

Documentación oficial de Anthropic para el diseño y construcción de agentes

Compilación de referencia (traducida) — Building Effective Agents, Agent SDK, Agent Skills y Tool Use

Compilado y traducido para Víctor Valotto — Laboratorio de Agentes (agentes-lab)

6 de agosto de 2026

Índice

Nota introductoria	3
1. Construyendo agentes efectivos	4
¿Qué son los agentes?	4
Cuándo (y cuándo no) usar agentes	4
Cuándo y cómo usar frameworks	4
Bloques de construcción, workflows y agentes	5
Combinando y personalizando estos patrones	7
Resumen	8
Apéndice 1: agentes en la práctica	8
Apéndice 2: ingeniería de prompts para tus herramientas	8
2. Agent SDK — Descripción general	10
Comparar el Agent SDK con otras herramientas de Claude	10
Capacidades	10
Primeros pasos	11
Changelog y reporte de bugs	11
Licencia y términos	11
Próximos pasos sugeridos por la documentación oficial	11
3. Agent Skills — Descripción general	12
Por qué usar Skills	12
Usando Skills	12
Cómo funcionan las Skills	12
Dónde funcionan las Skills	14
Estructura de una Skill	14
Consideraciones de seguridad	14
Skills disponibles	15
Limitaciones y restricciones	15
4. Uso de herramientas con Claude — Descripción general	16
Cómo funciona el uso de herramientas	16
Cuándo Claude usa herramientas	17
Elegir una herramienta	18
Precios	18

Nota introductoria

Este documento reúne, en un solo PDF, las cuatro piezas de documentación oficial de Anthropic más relevantes para el diseño y la construcción de agentes con Claude. Es una **traducción al español** del contenido publicado en anthropic.com y docs.claude.com, organizada para lectura offline y como material de referencia del laboratorio agentes-lab. Los términos técnicos que no tienen una traducción unívoca en la comunidad hispanohablante de desarrollo (*tool use*, *prompt*, *framework*, nombres de parámetros de API, etc.) se mantienen en inglés, igual que todo el código y los identificadores literales (`tool_result`, `stop_reason`, `SKILL.md`).

Conexión con la hoja de ruta del laboratorio (CLAUDE.md):

- **“Construyendo agentes efectivos”** (Anthropic, dic. 2024) — el documento fundacional. Define la distinción entre *workflows* y *agentes*, los patrones de composición (encadenamiento de prompts, enrutamiento, paralelización, orquestador-trabajadores, evaluador-optimizador) y los tres principios de diseño (simplicidad, transparencia, ACI). Insumo directo de la Etapa 1 (modelo mental agéntico) y referencia permanente para las Etapas 3 y 4.
- **Agent SDK — Descripción general** — documenta el mismo motor usado en `hola_agente_sdk/agente_ddd_sdk`. Útil para profundizar en hooks, subagentes, MCP y permisos antes de escalar a sistemas multi-agente (Etapa 4).
- **Agent Skills — Descripción general** — arquitectura de *divulgación progresiva* que explica cómo funcionan las skills que ya usás en este entorno (*escritura-valotto*, *libro-solid-maquetacion*, etc.) y que es insumo directo para diseñar los futuros GPTs/asistentes de IEC 62304.
- **Uso de herramientas con Claude (descripción general)** — la mecánica de *tool use* subyacente tanto en `hola_agente/agente_ddd.py` (API directa) como en el SDK. Explica herramientas de cliente vs. herramientas de servidor, cuándo Claude decide llamar una herramienta, y el costo en tokens de cada modelo.

Cada sección conserva sus enlaces originales a la documentación en línea, por si necesitás la versión más actualizada o navegar a páginas relacionadas no incluidas aquí.

1. Construyendo agentes efectivos

Fuente: anthropic.com/engineering/building-effective-agents **Publicado:** 19 de diciembre de 2024 — Anthropic Engineering **Autores:** Erik S. y Barry Zhang

Hemos trabajado con decenas de equipos que construyen agentes basados en LLM en distintas industrias. De forma consistente, las implementaciones más exitosas usan patrones simples y componibles en lugar de frameworks complejos.

Durante el último año, trabajamos con decenas de equipos que construyen agentes de modelos de lenguaje grande (LLM) en todas las industrias. De forma consistente, las implementaciones más exitosas no usaban frameworks complejos ni librerías especializadas. En cambio, construían con patrones simples y componibles.

En este artículo compartimos lo que aprendimos trabajando con nuestros clientes y construyendo agentes nosotros mismos, y damos consejos prácticos para desarrolladores que quieren construir agentes efectivos.

¿Qué son los agentes?

“Agente” puede definirse de varias formas. Algunos clientes definen a los agentes como sistemas totalmente autónomos que operan de forma independiente durante períodos extendidos, usando distintas herramientas para lograr tareas complejas. Otros usan el término para describir implementaciones más prescriptivas que siguen workflows predefinidos. En Anthropic, categorizamos todas estas variaciones como **sistemas agénticos**, pero trazamos una distinción arquitectónica importante entre **workflows** y **agentes**:

- Los **workflows** son sistemas donde los LLMs y las herramientas se orquestan a través de rutas de código predefinidas.
- Los **agentes**, en cambio, son sistemas donde los LLMs dirigen dinámicamente sus propios procesos y el uso de herramientas, manteniendo el control sobre cómo logran sus tareas.

Más abajo exploramos ambos tipos de sistemas agénticos en detalle. En el Apéndice 1 (“Agentes en la práctica”) describimos dos dominios donde los clientes encontraron particular valor en el uso de este tipo de sistemas.

Cuándo (y cuándo no) usar agentes

Al construir aplicaciones con LLMs, recomendamos encontrar la solución más simple posible, y aumentar la complejidad solo cuando sea necesario. Esto puede significar directamente no construir sistemas agénticos. Los sistemas agénticos suelen intercambiar latencia y costo por mejor desempeño en la tarea, y hay que considerar cuándo ese intercambio tiene sentido.

Cuando se justifica más complejidad, los workflows ofrecen previsibilidad y consistencia para tareas bien definidas, mientras que los agentes son la mejor opción cuando se necesita flexibilidad y toma de decisiones dirigida por el modelo a escala. Para muchas aplicaciones, sin embargo, suele alcanzar con optimizar llamadas individuales al LLM con recuperación de información (retrieval) y ejemplos en contexto.

Cuándo y cómo usar frameworks

Existen muchos frameworks que facilitan la implementación de sistemas agénticos, entre ellos:

- El [Claude Agent SDK](#);
- Strands Agents SDK de AWS;
- Rivet, un constructor visual de workflows de LLM tipo arrastrar y soltar; y
- Vellum, otra herramienta gráfica para construir y probar workflows complejos.

Estos frameworks facilitan arrancar rápido al simplificar tareas estándar de bajo nivel como llamar a los LLMs, definir y parsear herramientas, y encadenar llamadas. Sin embargo, a menudo crean capas extra

de abstracción que pueden ocultar los prompts y respuestas subyacentes, lo que dificulta el debugging. También pueden tentar a agregar complejidad cuando una configuración más simple alcanzaría.

Sugerimos que los desarrolladores empiecen usando las APIs de LLM directamente: muchos patrones se pueden implementar en pocas líneas de código. Si usás un framework, asegurate de entender el código subyacente. Los supuestos incorrectos sobre lo que hay “debajo del capó” son una fuente común de errores.

Ver el [cookbook de Anthropic](#) para implementaciones de ejemplo.

Bloques de construcción, workflows y agentes

En esta sección exploramos los patrones comunes para sistemas agénticos que vimos en producción. Empezamos con nuestro bloque de construcción fundamental — el LLM aumentado — y aumentamos progresivamente la complejidad, desde workflows composicionales simples hasta agentes autónomos.

Bloque de construcción: el LLM aumentado

El bloque de construcción básico de los sistemas agénticos es un LLM potenciado con aumentos como recuperación de información, herramientas y memoria. Nuestros modelos actuales pueden usar estas capacidades activamente — generando sus propias consultas de búsqueda, seleccionando las herramientas apropiadas, y determinando qué información retener.

Recomendamos enfocarse en dos aspectos clave de la implementación: adaptar estas capacidades a tu caso de uso específico, y asegurarte de que provean una interfaz sencilla y bien documentada para tu LLM. Si bien hay muchas formas de implementar estos aumentos, un enfoque es a través del [Model Context Protocol](#), que permite a los desarrolladores integrarse con un ecosistema creciente de herramientas de terceros con una implementación de cliente simple.

Para el resto de este artículo, asumimos que cada llamada al LLM tiene acceso a estas capacidades aumentadas.

Workflow: encadenamiento de prompts (prompt chaining)

El encadenamiento de prompts descompone una tarea en una secuencia de pasos, donde cada llamada al LLM procesa la salida de la anterior. Se pueden agregar verificaciones programáticas (“compuertas”) en cualquier paso intermedio para asegurar que el proceso siga en curso.

Cuándo usar este workflow: ideal para situaciones donde la tarea se puede descomponer fácil y limpiamente en subtarefas fijas. El objetivo principal es intercambiar latencia por mayor precisión, haciendo que cada llamada al LLM sea una tarea más simple.

Ejemplos donde el encadenamiento de prompts es útil:

- Generar textos de marketing y luego traducirlos a otro idioma.
- Escribir el esquema de un documento, verificar que el esquema cumple ciertos criterios, y luego escribir el documento a partir del esquema.

Workflow: enrutamiento (routing)

El enrutamiento clasifica una entrada y la dirige hacia una tarea de seguimiento especializada. Este workflow permite la separación de responsabilidades y la construcción de prompts más especializados. Sin este workflow, optimizar para un tipo de entrada puede perjudicar el desempeño en otros tipos.

Cuándo usar este workflow: funciona bien para tareas complejas donde hay categorías distintas que conviene manejar por separado, y donde la clasificación se puede hacer con precisión, ya sea por un LLM o por un modelo/algoritmo de clasificación más tradicional.

Ejemplos donde el enrutamiento es útil:

- Dirigir distintos tipos de consultas de atención al cliente (preguntas generales, solicitudes de reembolso, soporte técnico) hacia distintos procesos, prompts y herramientas.
- Enrutar preguntas fáciles/comunes a modelos más chicos y económicos como Claude Haiku 4.5, y preguntas difíciles/inusuales a modelos más capaces como Claude Sonnet 4.5, para optimizar el mejor desempeño.

Workflow: paralelización

Los LLMs a veces pueden trabajar simultáneamente en una tarea y tener sus salidas agregadas programáticamente. Este workflow, la paralelización, se manifiesta en dos variaciones clave:

- **Seccionamiento (sectioning):** dividir una tarea en subtareas independientes que corren en paralelo.
- **Votación (voting):** correr la misma tarea múltiples veces para obtener salidas diversas.

Cuándo usar este workflow: la paralelización es efectiva cuando las subtareas divididas se pueden paralelizar por velocidad, o cuando se necesitan múltiples perspectivas o intentos para obtener resultados con mayor confianza. Para tareas complejas con múltiples consideraciones, los LLMs generalmente se desempeñan mejor cuando cada consideración la maneja una llamada separada al LLM, permitiendo atención focalizada en cada aspecto específico.

Ejemplos donde la paralelización es útil:

- *Seccionamiento:*
 - Implementar barreras de seguridad (guardrails) donde una instancia del modelo procesa las consultas del usuario mientras otra las filtra por contenido o solicitudes inapropiadas. Esto tiende a funcionar mejor que hacer que la misma llamada al LLM maneje tanto las barreras de seguridad como la respuesta principal.
 - Automatizar evaluaciones (evals) para medir el desempeño del LLM, donde cada llamada evalúa un aspecto distinto del desempeño del modelo sobre un prompt dado.
- *Votación:*
 - Revisar una porción de código en busca de vulnerabilidades, donde varios prompts distintos revisan y marcan el código si encuentran un problema.
 - Evaluar si un contenido dado es inapropiado, con múltiples prompts evaluando distintos aspectos o requiriendo distintos umbrales de votación para equilibrar falsos positivos y negativos.

Workflow: orquestador-trabajadores

En el workflow de orquestador-trabajadores, un LLM central descompone dinámicamente las tareas, las delega a LLMs trabajadores, y sintetiza sus resultados.

Cuándo usar este workflow: bien adaptado para tareas complejas donde no se pueden predecir las subtareas necesarias (en programación, por ejemplo, la cantidad de archivos que hay que cambiar y la naturaleza del cambio en cada archivo probablemente dependan de la tarea). Aunque es topográficamente similar a la paralelización, la diferencia clave es su flexibilidad — las subtareas no están predefinidas, sino que las determina el orquestador según la entrada específica.

Ejemplo donde orquestador-trabajadores es útil:

- Productos de programación que hacen cambios complejos en múltiples archivos cada vez.
- Tareas de búsqueda que implican recopilar y analizar información de múltiples fuentes en busca de información posiblemente relevante.

Workflow: evaluador-optimizador

En el workflow de evaluador-optimizador, una llamada al LLM genera una respuesta mientras otra provee evaluación y feedback en un ciclo.

Cuándo usar este workflow: particularmente efectivo cuando hay criterios de evaluación claros, y cuando el refinamiento iterativo aporta valor medible. Las dos señales de buen ajuste son, primero, que las respuestas del LLM se puedan mejorar demostrablemente cuando un humano articula su feedback; y segundo, que el LLM pueda proveer ese feedback. Esto es análogo al proceso de escritura iterativa por el que podría pasar un escritor humano al producir un documento pulido.

Ejemplos donde evaluador-optimizador es útil:

- Traducción literaria, donde hay matices que el LLM traductor podría no captar inicialmente, pero donde un LLM evaluador puede dar críticas útiles.
- Tareas de búsqueda complejas que requieren múltiples rondas de búsqueda y análisis para reunir información integral, donde el evaluador decide si se justifican más búsquedas.

Agentes

Los agentes están emergiendo en producción a medida que los LLMs maduran en capacidades clave: comprender entradas complejas, razonar y planificar, usar herramientas de forma confiable, y recuperarse de errores. Los agentes comienzan su trabajo a partir de un comando del usuario humano, o de una discusión interactiva con él. Una vez que la tarea está clara, los agentes planifican y operan de forma independiente, potencialmente volviendo al humano para más información o criterio. Durante la ejecución, es crucial que los agentes obtengan “verdad de terreno” (ground truth) del entorno en cada paso (como resultados de llamadas a herramientas o ejecución de código) para evaluar su progreso. Los agentes pueden entonces pausar para recibir feedback humano en puntos de control o al encontrar bloqueos. La tarea suele terminar al completarse, pero también es común incluir condiciones de parada (como un número máximo de iteraciones) para mantener el control.

Los agentes pueden manejar tareas sofisticadas, pero su implementación suele ser directa. Por lo general son simplemente LLMs que usan herramientas basándose en el feedback del entorno, en un ciclo. Por eso es crucial diseñar los conjuntos de herramientas y su documentación de forma clara y cuidadosa (ver Apéndice 2, “Ingeniería de prompts para tus herramientas”).

Cuándo usar agentes: los agentes se pueden usar para problemas abiertos donde es difícil o imposible predecir la cantidad de pasos necesarios, y donde no se puede fijar una ruta de antemano. El LLM potencialmente va a operar durante muchos turnos, y hay que tener cierto nivel de confianza en su toma de decisiones. La autonomía de los agentes los hace ideales para escalar tareas en entornos de confianza.

La naturaleza autónoma de los agentes implica mayores costos, y el potencial de errores que se acumulan. Recomendamos pruebas extensivas en entornos aislados (sandbox), junto con las barreras de seguridad apropiadas.

Ejemplos donde los agentes son útiles (de las propias implementaciones de Anthropic):

- Un agente de programación para resolver tareas de SWE-bench, que implican ediciones a muchos archivos a partir de una descripción de tarea;
- La implementación de referencia de “uso de computadora” (*computer use*), donde Claude usa una computadora para lograr tareas.

Combinando y personalizando estos patrones

Estos bloques de construcción no son prescriptivos. Son patrones comunes que los desarrolladores pueden moldear y combinar para adaptarse a distintos casos de uso. La clave del éxito, como con cualquier función de LLM, es medir el desempeño e iterar sobre las implementaciones. Para repetirlo: hay que considerar agregar complejidad *solo* cuando mejora demostrablemente los resultados.

Resumen

El éxito en el espacio de los LLM no se trata de construir el sistema más sofisticado. Se trata de construir el sistema *correcto* para tus necesidades. Empezá con prompts simples, optimízalos con evaluación integral, y agregá sistemas agénticos de múltiples pasos solo cuando las soluciones más simples no alcancen.

Al implementar agentes, en Anthropic seguimos tres principios centrales:

1. Mantener la **simplicidad** en el diseño del agente.
2. Priorizar la **transparencia**, mostrando explícitamente los pasos de planificación del agente.
3. Diseñar cuidadosamente la interfaz agente-computadora (ACI, por *agent-computer interface*) a través de **documentación y pruebas** exhaustivas de las herramientas.

Los frameworks pueden ayudar a arrancar rápido, pero no hay que dudar en reducir capas de abstracción y construir con componentes básicos a medida que se avanza hacia producción. Siguiendo estos principios, se pueden crear agentes que no solo sean poderosos, sino también confiables, mantenibles y dignos de la confianza de sus usuarios.

Apéndice 1: agentes en la práctica

El trabajo de Anthropic con clientes reveló dos aplicaciones particularmente prometedoras para agentes de IA que demuestran el valor práctico de los patrones discutidos arriba. Ambas aplicaciones ilustran cómo los agentes agregan más valor en tareas que requieren tanto conversación como acción, tienen criterios de éxito claros, habilitan ciclos de feedback, e integran supervisión humana significativa.

A. Atención al cliente

La atención al cliente combina interfaces de chatbot familiares con capacidades potenciadas mediante integración de herramientas. Es un ajuste natural para agentes más abiertos porque:

- Las interacciones de soporte naturalmente siguen un flujo conversacional mientras requieren acceso a información y acciones externas;
- Se pueden integrar herramientas para traer datos del cliente, historial de pedidos y artículos de la base de conocimiento;
- Acciones como emitir reembolsos o actualizar tickets se pueden manejar programáticamente; y
- El éxito se puede medir claramente a través de resoluciones definidas por el usuario.

B. Agentes de programación

El espacio del desarrollo de software mostró un potencial notable para las funciones de LLM, con capacidades que evolucionaron desde el autocompletado de código hasta la resolución autónoma de problemas. Los agentes son particularmente efectivos porque:

- Las soluciones de código son verificables mediante pruebas automatizadas;
- Los agentes pueden iterar sobre soluciones usando los resultados de las pruebas como feedback;
- El espacio del problema está bien definido y estructurado; y
- La calidad de la salida se puede medir objetivamente.

En la implementación propia de Anthropic, los agentes pueden resolver issues reales de GitHub en el benchmark SWE-bench Verified basándose únicamente en la descripción del pull request. Sin embargo, mientras las pruebas automatizadas ayudan a verificar la funcionalidad, la revisión humana sigue siendo crucial para asegurar que las soluciones se alineen con los requerimientos más amplios del sistema.

Apéndice 2: ingeniería de prompts para tus herramientas

Sin importar qué sistema agéntico estés construyendo, las herramientas probablemente sean una parte importante de tu agente. Las herramientas permiten a Claude interactuar con servicios y APIs externas

especificando su estructura y definición exacta en la API. Cuando Claude responde, va a incluir un bloque de uso de herramienta (*tool use*) en la respuesta de la API si planea invocar una herramienta. Las definiciones y especificaciones de herramientas merecen tanta atención de ingeniería de prompts como los prompts generales.

A menudo hay varias formas de especificar la misma acción. Por ejemplo, se puede especificar una edición de archivo escribiendo un diff, o reescribiendo el archivo completo. Para salida estructurada, se puede devolver código dentro de markdown o dentro de JSON. En ingeniería de software, diferencias como estas son cosméticas y se pueden convertir de una a otra sin pérdida. Sin embargo, algunos formatos son mucho más difíciles de escribir para un LLM que otros. Escribir un diff requiere saber cuántas líneas están cambiando en el encabezado del bloque antes de escribir el código nuevo. Escribir código dentro de JSON (comparado con markdown) requiere escapar de más saltos de línea y comillas.

Sugerencias de Anthropic para decidir formatos de herramientas:

- Darle al modelo suficientes tokens para “pensar” antes de que se escriba a sí mismo hacia un callejón sin salida.
- Mantener el formato cercano a lo que el modelo vio naturalmente en textos de internet.
- Asegurarse de que no haya “sobrecarga” de formato, como tener que mantener un conteo preciso de miles de líneas de código, o escapar strings de cualquier código que escriba.

Una regla general es pensar en cuánto esfuerzo se invierte en interfaces humano-computadora (HCI), y planear invertir el mismo esfuerzo en crear buenas interfaces *agente*-computadora (ACI):

- Ponete en el lugar del modelo. ¿Es obvio cómo usar esta herramienta, a partir de la descripción y los parámetros, o habría que pensarlo con cuidado? Si es así, probablemente también lo sea para el modelo. Una buena definición de herramienta suele incluir ejemplos de uso, casos límite, requerimientos de formato de entrada, y límites claros respecto de otras herramientas.
- ¿Cómo se pueden cambiar los nombres o descripciones de parámetros para que las cosas sean más obvias? Pensalo como escribir un buen docstring para un desarrollador junior de tu equipo. Esto es especialmente importante cuando se usan muchas herramientas similares.
- Probá cómo el modelo usa tus herramientas: corré muchas entradas de ejemplo en el [workbench de Anthropic](#) para ver qué errores comete el modelo, e iterá.
- Aplicá *poka-yoke* a tus herramientas — cambiá los argumentos para que sea más difícil cometer errores.

Mientras construían el agente para SWE-bench, en Anthropic pasaron más tiempo optimizando las herramientas que el prompt general. Por ejemplo, encontraron que el modelo cometía errores con herramientas que usaban rutas de archivo relativas después de que el agente se movía fuera del directorio raíz. Para solucionarlo, cambiaron la herramienta para requerir siempre rutas absolutas — y encontraron que el modelo usó este método sin fallas.

2. Agent SDK — Descripción general

Fuente: docs.claude.com/en/docs/agent-sdk/overview

Un agente es una aplicación que completa una tarea planificando sus propios pasos y llamando a herramientas que leen archivos, ejecutan comandos, o editan código. El Agent SDK te da las mismas herramientas, el mismo *agent loop*, y la misma gestión de contexto que potencian a Claude Code, programables en Python y TypeScript.

Comparar el Agent SDK con otras herramientas de Claude

Si estás...	Usá	Por qué
Construyendo un agente sin implementar el tool loop vos mismo	Agent SDK	Librería que corre el loop del agente en tu propio proceso, en Python o TypeScript.
Haciendo desarrollo interactivo o tareas puntuales desde una terminal	Claude Code CLI	La interfaz de terminal, pensada para uso diario interactivo.
Llamando la API directamente e implementando el tool loop vos mismo	Client SDK	Acceso directo a la API de Anthropic en lugar de a Claude Code. Vos implementás el tool loop.
Corriendo agentes de larga duración o asíncronos sin gestionar tu propio sandbox	Managed Agents	API REST hospedada; Anthropic corre el agente y el sandbox.

El SDK está disponible como librería solo para Python y TypeScript. Para manejar el mismo agent loop desde otro lenguaje, se puede correr el CLI como subprocesso con el flag `-p` y `--output-format json`.

Capacidades

Capacidad	Qué hace
Herramientas integradas	Leer, escribir, editar archivos, ejecutar comandos, y buscar en la web
Hooks	Ejecutar código propio en puntos clave del ciclo de vida del agente
Subagentes	Generar agentes especializados para subtareas puntuales
MCP	Conectar herramientas y fuentes de datos externas vía el Model Context Protocol
Permisos	Controlar qué herramientas corren automáticamente y cuáles necesitan aprobación
Sesiones	Mantener contexto entre intercambios, retomar o bifurcar más tarde
Skills, comandos, memoria	Se cargan automáticamente desde <code>.claude/</code> del proyecto y desde <code>~/ .claude/</code> , igual que en Claude Code
Plugins	Empaquetar skills, agentes, hooks y servidores MCP, y cargarlos por ruta local

Primeros pasos

Seguir el Quickstart oficial para instalar el SDK, configurar la API key, y construir el primer agente (uno que encuentra y arregla bugs en código existente).

Salvo aprobación previa, Anthropic no permite que desarrolladores externos ofrezcan login de claude.ai o los rate limits de claude.ai en sus propios productos, incluyendo agentes construidos con el Claude Agent SDK. Usar los métodos de autenticación por API key descritos en el Quickstart.

Changelog y reporte de bugs

- **TypeScript SDK:** [CHANGELOG.md](#) / [issues](#)
- **Python SDK:** [CHANGELOG.md](#) / [issues](#)

Licencia y términos

El uso del Claude Agent SDK se rige por los Commercial Terms of Service de Anthropic, incluso cuando se usa para productos y servicios ofrecidos a clientes propios, salvo que un componente o dependencia específica esté cubierta por otra licencia indicada en su archivo LICENSE.

Próximos pasos sugeridos por la documentación oficial

- **Quickstart:** construir el primer agente que encuentra y arregla bugs.
- **Agent loop:** cómo Claude planifica, llama herramientas, y decide cuándo una tarea está terminada.
- **Example agents:** apps de demostración para desarrollo local.
- **TypeScript SDK / Python SDK:** referencia completa de API y ejemplos.
- **Diseño del harness del agente:** cómo el equipo de Claude Code usa workflows dinámicos para orquestar subagentes a escala.

3. Agent Skills — Descripción general

Fuente: docs.claude.com/en/docs/agents-and-tools/agent-skills/overview

Las Agent Skills son capacidades modulares que extienden la funcionalidad de Claude. Cada Skill empaqueta instrucciones, metadatos, y recursos opcionales (scripts, plantillas) que Claude usa automáticamente cuando son relevantes.

Por qué usar Skills

Las Skills son recursos reutilizables basados en filesystem que le dan a Claude experiencia específica de dominio: workflows, contexto y mejores prácticas que convierten a un agente generalista en un especialista. A diferencia de los prompts (instrucciones a nivel de conversación para tareas puntuales), las Skills se cargan bajo demanda, así que no hace falta repetir la misma guía en cada conversación.

Beneficios clave:

- **Especializar a Claude:** adaptar capacidades para tareas específicas de dominio.
- **Reducir la repetición:** crear una vez, usar automáticamente.
- **Componer capacidades:** combinar Skills para tareas complejas de múltiples pasos.

Usando Skills

Anthropic provee Agent Skills pre-construidas para tareas comunes con documentos (PowerPoint, Excel, Word, PDF), y podés crear tus propias Skills personalizadas. Ambas funcionan de la misma forma: una vez que una Skill está disponible en tu entorno, Claude la usa automáticamente cuando es relevante para tu pedido.

Las Agent Skills pre-construidas están disponibles en claude.ai, la Claude API, Claude Platform on AWS, y Microsoft Foundry.

Las Skills personalizadas te permiten empaquetar experiencia de dominio y conocimiento organizacional. Están disponibles en todos los productos de Claude: se crean en Claude Code, se suben a través de la Claude API, o se agregan en la configuración de claude.ai.

Cómo funcionan las Skills

Las Skills usan el entorno de máquina virtual (VM) de Claude para proveer capacidades que van más allá de lo posible solo con prompts. Claude opera en una máquina virtual con acceso al filesystem, lo que permite que las Skills existan como directorios que contienen instrucciones, código ejecutable, y materiales de referencia, organizados como una guía de onboarding que crearías para un nuevo miembro del equipo.

Esta arquitectura basada en filesystem habilita la **divulgación progresiva** (*progressive disclosure*): Claude carga información por etapas según la necesita, en lugar de consumir contexto por adelantado.

Las Skills pueden contener tres tipos de contenido, cada uno cargado en un momento distinto:

Nivel 1: Metadatos (siempre cargados)

El frontmatter YAML de la Skill provee información de descubrimiento:

```
---
name: pdf-processing
description: Extract text and tables from PDF files, fill forms, merge documents.
            Use when working with PDF files or when the user mentions PDFs, forms, or
            document extraction.
---
```

Claude carga estos metadatos al inicio y los incluye en el system prompt. El `description` es lo que Claude compara contra tu pedido para determinar si dispara la Skill, así que debe decir tanto qué hace la Skill como cuándo usarla. Este enfoque liviano significa que se pueden instalar muchas Skills sin penalidad de contexto: hasta que una Skill se dispara, solo su nombre y descripción ocupan contexto.

Nivel 2: Instrucciones (cargadas al dispararse)

El cuerpo principal de `SKILL.md` contiene conocimiento procedimental: workflows, mejores prácticas y guías. Cuando el pedido del usuario coincide con la descripción de una Skill, Claude lee `SKILL.md` del filesystem usando `bash`. Recién ahí ese contenido entra a la ventana de contexto.

Nivel 3: Recursos y código (cargados según necesidad)

Las Skills pueden empaquetar materiales adicionales:

```
pdf-processing/  
├── SKILL.md (instrucciones principales)  
├── FORMS.md (guía de llenado de formularios)  
├── REFERENCE.md (referencia detallada de API)  
└── scripts/  
    └── fill_form.py (script utilitario)
```

- **Instrucciones:** archivos markdown adicionales con guías especializadas.
- **Código:** scripts ejecutables que Claude corre usando `bash`, sin cargar su código al contexto.
- **Recursos:** materiales de referencia como esquemas de base de datos, documentación de API, plantillas o ejemplos.

Nivel	Cuándo se carga	Costo en tokens	Contenido
Nivel 1: Metadatos	Siempre (al inicio)	~100 tokens por Skill	name y description del frontmatter YAML
Nivel 2: Instrucciones	Cuando la Skill se dispara	Menos de 5k tokens	Cuerpo de <code>SKILL.md</code>
Nivel 3+: Recursos	Según necesidad	Ninguno hasta acceder	Archivos empaquetados; scripts corren vía <code>bash</code>

La arquitectura de las Skills

Las Skills corren en un entorno de ejecución de código donde Claude tiene acceso al filesystem, comandos `bash`, y capacidades de ejecución de código. Las Skills existen como directorios en una máquina virtual, y Claude interactúa con ellas usando los mismos comandos `bash` que usarías para navegar archivos en tu computadora.

Lo que esta arquitectura habilita:

- **Acceso a archivos bajo demanda:** Claude lee solo los archivos que cada tarea necesita.
- **Ejecución eficiente de scripts:** cuando Claude corre un script, su código nunca entra al contexto — solo su salida.
- **Sin límite práctico de contenido empaquetado:** los archivos no consumen contexto hasta que se acceden, así que las Skills pueden incluir documentación extensa sin penalidad si no se usa.

Ejemplo: cargando una Skill de procesamiento de PDF

1. **Inicio:** el system prompt incluye: `pdf-processing - Extract text and tables from PDF files... Use when working with PDF files...`

2. **Pedido del usuario:** “Extraé el texto de este PDF y resumilo.”
3. **Claude invoca:** `bash: cat pdf-processing/SKILL.md → instrucciones cargadas al contexto.`
4. **Claude determina:** no se necesita llenar formularios, así que `FORMS.md` no se lee.
5. **Claude ejecuta:** usa las instrucciones de `SKILL.md` para completar la tarea.

Dónde funcionan las Skills

- **Claude API:** soporta Skills pre-construidas y personalizadas vía el parámetro `container` y el `skill_id`. Requiere la herramienta de ejecución de código y el header beta `skills-2025-10-02`. Corren en un contenedor aislado (sandboxed) sin acceso a red ni instalación de paquetes en runtime.
- **Claude Code:** soporta Skills personalizadas como directorios con `SKILL.md`, basadas en filesystem (`~/.claude/skills/` personal o `.claude/skills/` de proyecto), sin necesidad de subirlas vía API.
- **claude.ai:** soporta Skills pre-construidas (activas al crear documentos) y Skills personalizadas (subidas como zip en Settings > Features, disponibles en planes Pro/Max/Team/Enterprise).

Estructura de una Skill

Toda Skill requiere un archivo `SKILL.md` con frontmatter YAML:

```
---
name: your-skill-name
description: Brief description of what this Skill does and when to use it
---

# Your Skill Name

## Instructions
[Guía clara, paso a paso, para que Claude la siga]

## Examples
[Ejemplos concretos de uso de esta Skill]
```

Campos requeridos: name y description.

name: máximo 64 caracteres; solo minúsculas, números y guiones; sin tags XML; no puede contener palabras reservadas (“anthropic”, “claude”).

description: no vacío; máximo 1024 caracteres; sin tags XML. Debe incluir tanto qué hace la Skill como cuándo Claude debería usarla.

Consideraciones de seguridad

Usar Skills solo de fuentes confiables: las que creaste vos mismo o las que provee Anthropic. Las Skills dan a Claude nuevas capacidades a través de instrucciones y código, lo que también significa que una Skill maliciosa puede dirigir a Claude a invocar herramientas o ejecutar código de formas que no coinciden con el propósito declarado de la Skill.

Consideraciones clave de seguridad:

- **Auditar exhaustivamente:** revisar todos los archivos empaquetados (`SKILL.md`, scripts, imágenes, otros recursos), buscando patrones inusuales.
- **Las fuentes externas son riesgosas:** Skills que traen datos de URLs externas son particularmente riesgosas, ya que el contenido obtenido puede contener instrucciones maliciosas.
- **Mal uso de herramientas:** Skills maliciosas pueden invocar herramientas (operaciones de archivos, comandos bash, ejecución de código) de forma dañina.

- **Exposición de datos:** Skills con acceso a datos sensibles podrían diseñarse para filtrar información a sistemas externos.
- **Tratarlas como instalar software:** especial cuidado al integrar Skills en sistemas de producción con acceso a datos sensibles u operaciones críticas.

Skills disponibles

Agent Skills pre-construidas: PowerPoint (pptx), Excel (xlsx), Word (docx), PDF (pdf). Disponibles en la Claude API, Claude Platform on AWS, Microsoft Foundry y claude.ai.

Skills open-source: Anthropic también publica Skills open-source en el [repositorio de skills](#), incluyendo el *Claude API skill* que provee referencia de API actualizada, documentación de SDK y mejores prácticas para ocho lenguajes de programación.

Limitaciones y restricciones

Disponibilidad entre superficies: las Skills personalizadas no se sincronizan entre superficies. Una Skill subida a claude.ai debe subirse por separado a la API; las de Claude Code son independientes de ambas (basadas en filesystem).

Alcance de uso compartido:

- claude.ai: solo el usuario individual, cada miembro del equipo debe subir la suya.
- Claude API: a nivel workspace, todos los miembros acceden a las Skills subidas.
- Claude Code: personal (~/.claude/skills/) o de proyecto (.claude/skills/); también distribuible vía Claude Code Plugins.

Restricciones del entorno de ejecución:

- claude.ai: acceso a red variable según configuración de usuario/admin.
- Claude API: sin acceso a red, sin instalación de paquetes en runtime, solo dependencias pre-configuradas.
- Claude Code: acceso completo a red (igual que cualquier programa en la máquina del usuario); se desaconseja la instalación global de paquetes.

4. Uso de herramientas con Claude — Descripción general

Fuente: docs.claude.com/en/docs/agents-and-tools/tool-use/overview

El uso de herramientas (*tool use*) le permite a Claude llamar funciones que vos definís o que provee Anthropic. Claude determina cuándo llamar a una herramienta basándose en el pedido del usuario y la descripción de la herramienta. Luego devuelve una llamada estructurada que tu aplicación ejecuta (herramientas de cliente) o que Anthropic ejecuta (herramientas de servidor).

Ejemplo mínimo con una herramienta de servidor (búsqueda web), que Anthropic ejecuta por vos:

```
client = anthropic.Anthropic()
response = client.messages.create(
    model="claude-opus-5",
    max_tokens=1024,
    tools=[{"type": "web_search_20260209", "name": "web_search"}],
    messages=[{"role": "user", "content": "What's the latest on the Mars rover?"}],
)
print(response.content)
```

Claude corre la búsqueda en la infraestructura de Anthropic y devuelve los resultados citados en la misma respuesta. Para que Claude llame una función que vos definís, se pasa una herramienta con un `input_schema`, y luego se ejecuta la llamada cuando Claude devuelve un bloque `tool_use`.

Cómo funciona el uso de herramientas

Las herramientas difieren principalmente en dónde se ejecuta el código.

- Las **herramientas de cliente** (incluidas las herramientas definidas por el usuario y las herramientas con schema definido por Anthropic, como `bash` y `text_editor`) corren en tu aplicación. Claude responde con `stop_reason: "tool_use"` y uno o más bloques `tool_use`. Tu código ejecuta la operación y devuelve un `tool_result`.
- Las **herramientas de servidor** (como `web_search`, `web_fetch`, `code_execution` y `tool_search`) corren en la infraestructura de Anthropic: ves los resultados directamente sin manejar la ejecución.

Round trip completo para una herramienta de cliente (`get_weather`):

```
client = anthropic.Anthropic()

tools = [
    {
        "name": "get_weather",
        "description": "Get the current weather for a given location.",
        "input_schema": {
            "type": "object",
            "properties": {
                "location": {
                    "type": "string",
                    "description": "City and state, e.g. San Francisco, CA",
                }
            },
            "required": ["location"],
        },
    },
]

messages = [{"role": "user", "content": "What's the weather in San Francisco?"}]
```



```

# Claude responde con un bloque tool_use nombrando la herramienta y sus argumentos.
response = client.messages.create(
    model="claude-opus-5",
    max_tokens=1024,
    tools=tools,
    tool_choice={"type": "auto", "disable_parallel_tool_use": True},
    messages=messages,
)
tool_use = next(block for block in response.content if block.type == "tool_use")
print(f"Claude called {tool_use.name} with {json.dumps(tool_use.input)}")

# Ejecutar la herramienta, y devolver el resultado en un bloque tool_result.
weather = "15 degrees Celsius, partly cloudy"
messages += [
    {"role": "assistant", "content": response.content},
    {
        "role": "user",
        "content": [
            {"type": "tool_result", "tool_use_id": tool_use.id, "content": weather}
        ],
    },
]
followup = client.messages.create(
    model="claude-opus-5",
    max_tokens=1024,
    tools=tools,
    tool_choice={"type": "auto", "disable_parallel_tool_use": True},
    messages=messages,
)

# Claude usa el resultado para responder la pregunta original.
final_text = next(block for block in followup.content if block.type == "text")
print(final_text.text)

```

Salida:

```

Claude called get_weather with {"location": "San Francisco, CA"}
The current weather in San Francisco is 15 degrees Celsius with partly cloudy skies.

```

Para evitar escribir este round trip a mano, existe **Tool Runner**: los SDKs ejecutan tus herramientas y devuelven los resultados automáticamente.

Cuándo Claude usa herramientas

Con el `tool_choice` por defecto de `{"type": "auto"}`, Claude determina en cada turno si llamar una herramienta o responder directamente. Llama a una herramienta cuando el pedido coincide con la capacidad descrita de esa herramienta y la respuesta no está ya en el contexto. Responde directamente para conocimiento estable, tareas creativas y turnos conversacionales.

Este límite es ajustable a través del system prompt. Si Claude no está llamando herramientas cuando se espera, una instrucción liviana como "Use the tools to investigate before responding." aumenta el uso de herramientas. Una forma más fuerte como "Always call a tool first before responding." empuja más. Al contrario, "Use your judgment about whether to call a tool or respond directly." mantiene un comportamiento de disparo conservador.

Para forzar una llamada de herramienta en lugar de depender del prompting, se configura `tool_choice`.

Garantizar conformidad de schema con strict tool use: agregar `strict: true` a las definiciones de herramientas personalizadas asegura que las llamadas de Claude siempre coincidan exactamente con tu schema.

Elegir una herramienta

Herramientas propias: vos escribís el schema y tu aplicación ejecuta cada llamada — *Definir herramientas*, *Manejar llamadas a herramientas*.

Herramientas de cliente con schema de Anthropic: Anthropic publica el schema y entrena a Claude en él; tu aplicación sigue ejecutando cada llamada y devolviendo el `tool_result` — *Memory tool* (almacenar y recuperar información entre conversaciones en archivos que vos controlás), *Bash tool* (correr comandos de shell en una sesión persistente con estado), *Text editor tool* (ver y modificar archivos de texto), *Computer use tool* (tomar screenshots y controlar mouse/teclado en un entorno de escritorio).

Herramientas de servidor (corren en infraestructura de Anthropic, sin código handler en tu aplicación): *Web search tool*, *Web fetch tool*, *Code execution tool* (correr Python y bash en un contenedor aislado), *Advisor tool* (un modelo ejecutor más rápido consulta a un modelo asesor de mayor inteligencia a mitad de generación), *Tool search tool* (trabajar con miles de herramientas descubriéndolas y cargándolas bajo demanda), *MCP connector* (conectar a servidores MCP remotos sin un cliente MCP separado).

Precios

El uso de herramientas se cotiza en base a:

1. El total de tokens de entrada enviados al modelo (incluyendo el parámetro `tools`).
2. El número de tokens de salida generados.
3. Para herramientas del lado del servidor, precio adicional basado en uso (por ejemplo, *web search* cobra por búsqueda realizada).

Cuando usás `tools`, la API automáticamente incluye un system prompt especial que habilita el uso de herramientas. Tabla de tokens de ese system prompt por modelo (asumiendo al menos 1 herramienta provista; si no se proveen herramientas, `tool_choice: none` usa 0 tokens adicionales):

Modelo	auto / none	any / tool
Claude Opus 5	286 tokens	406 tokens
Claude Opus 4.8	290 tokens	410 tokens
Claude Opus 4.7	675 tokens	804 tokens
Claude Opus 4.6	497 tokens	589 tokens
Claude Opus 4.5	496 tokens	588 tokens
Claude Sonnet 5	354 tokens	474 tokens
Claude Sonnet 4.6	497 tokens	589 tokens
Claude Sonnet 4.5	496 tokens	588 tokens
Claude Haiku 4.5	496 tokens	588 tokens

Estos conteos se suman a los tokens de entrada/salida normales para calcular el costo total de una petición.

Fuentes

- [Building Effective Agents — Anthropic Engineering](#)
- [Agent SDK Overview — Claude Docs](#)
- [Agent Skills Overview — Claude Platform Docs](#)
- [Tool Use with Claude — Overview — Claude Platform Docs](#)

Compilado y traducido el 6 de agosto de 2026. El contenido pertenece a Anthropic PBC y se reproduce aquí, traducido, con fines de referencia personal dentro del laboratorio agentes - lab. Para la versión oficial más actualizada en inglés, consultar siempre los enlaces originales — la documentación de Claude cambia con frecuencia.